

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Operating Systems

COURSE CODE: CSA-0404

COURSE OUTCOME COVERED

CO3: Evaluate memory management mechanisms such as linking, paging, and segmentation to determine their effectiveness for different system requirements

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks: 100

CASE STUDY:

1.

Describe the complete **address translation mechanism using base and limit registers** in the given system. Explain how this method ensures memory protection and prevents processes from accessing unauthorized memory locations.

2.

Explain the concept of **paging** as implemented in the system. Describe how logical memory is divided into pages and mapped to physical frames. Discuss how paging helps in avoiding external fragmentation and improving memory utilization.

3.

Describe the working of the **Translation Lookaside Buffer (TLB)** in the given system. Explain how TLB reduces memory access time and improves overall system performance.

4.

Differentiate between **TLB hit and TLB miss** in detail. Explain how each case affects memory access time and system efficiency, and illustrate with an example.

5.

Describe the concept of **segmentation** in detail. Explain how programs are divided into logical segments such as code, data, and stack, and discuss the benefits of segmentation over other memory management techniques.

6.

Explain the **structure and functioning of a segment table**. Describe how logical addresses are translated into physical addresses using segment number and offset.

7.

Compare and contrast **paging and segmentation** in detail. Discuss their differences in terms of memory allocation, fragmentation, flexibility, and performance.

8.

Explain the concept of **virtual memory** and its importance in modern operating systems. Describe how it allows execution of programs larger than the available physical memory.

9.

Describe the **page fault handling process** in detail. Explain each step involved when a page is not found in memory and how the operating system resolves the fault.

10.

Explain the need for **page replacement algorithms** in virtual memory systems. Describe the working of the **FIFO page replacement algorithm** with a suitable example.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Case	25	Thorough understanding ; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding ; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

1. Address Translation Using Base and Limit Registers

Answer

The **base register** stores the starting physical address of a process, while the **limit register** specifies the size of its valid memory area.

For a logical address:

$$\text{Physical Address} = \text{Base} + \text{Logical Address}$$

The logical address is checked against the limit. If:

$$0 \leq \text{Logical Address} < \text{Limit}$$

the access is allowed. Otherwise, it is rejected.

Example:

Base = 1000, Limit = 500, Logical Address = 200

$$\text{Physical Address} = 1000 + 200 = \mathbf{1200}$$

Thus, base and limit registers provide **memory protection** by preventing unauthorized access.

2. Paging

Answer

Paging divides logical memory into fixed-size **pages** and physical memory into fixed-size **frames**.

A **page table** maps each page to a frame.

$$\text{Physical Address} = \text{Frame Number} \times \text{Page Size} + \text{Offset}$$

Paging allows pages to be placed in any available frames. Therefore, continuous physical memory is not required.

Advantages

- Eliminates external fragmentation.
- Improves memory utilization.
- Allows non-contiguous memory allocation.
- Supports virtual memory.

3. Translation Lookaside Buffer (TLB)

Answer

TLB is a small, high-speed cache that stores recently used **page number** → **frame number** mappings.

When the CPU generates a logical address:

1. TLB is checked first.
2. If the page is found, it is a **TLB hit**.
3. If not found, it is a **TLB miss** and the page table is accessed.
4. The physical address is then obtained.

TLB reduces the number of page-table accesses and therefore **improves memory access speed**.

4. TLB Hit vs TLB Miss

TLB Hit	TLB Miss
Page is found in TLB	Page is not found in TLB
Faster access	Slower access
Page table need not be accessed	Page table must be accessed
Improves performance	Reduces performance

Example

Suppose TLB access = 10 ns and memory access = 100 ns.

TLB Hit:

$$10 + 100 = \mathbf{110 \text{ ns}}$$

TLB Miss:

$$10 + 100 + 100 = \mathbf{210 \text{ ns}}$$

Therefore, a higher TLB hit ratio improves system performance.

5. Segmentation

Answer

Segmentation divides a program into **logical segments** based on its structure.

Common segments are:

- **Code segment** – contains program instructions.
- **Data segment** – contains variables.

- **Stack segment** – contains function calls and local variables.

A logical address consists of:

Segment Number + Offset

Advantages

- Represents the logical structure of a program.
- Provides protection for individual segments.
- Supports sharing of code and data.
- Segment sizes can be different.

Segmentation is more flexible than fixed-size paging for representing program structure.

6. Segment Table

Answer

A **segment table** contains information about each segment, mainly:

- **Base:** Starting physical address.
- **Limit:** Size of the segment.

Logical address contains:

Segment Number + Offset

Translation steps:

1. Use segment number to find the segment table entry.
2. Check whether $\text{Offset} < \text{Limit}$.
3. If valid:

Physical Address = Base + Offset

4. Otherwise, a segmentation fault occurs.

Example

Base	=	5000
------	---	------

Limit	=	1000
-------	---	------

Offset = 300

Physical Address = $5000 + 300 = \mathbf{5300}$

7. Paging vs Segmentation

Paging	Segmentation
Divides memory into fixed-size pages	Divides program into logical segments
Pages have equal size	Segments have different sizes
Uses page table	Uses segment table
Eliminates external fragmentation	Can cause external fragmentation
Less logical from programmer's view	Matches program structure
Good for efficient memory allocation	Good for protection and sharing

Conclusion: Paging is mainly used for efficient physical memory management, while segmentation provides a logical view of a program.

8. Virtual Memory

Answer

Virtual memory allows a program to execute even when the complete program cannot fit into physical RAM.

Only required pages are loaded into memory. The remaining pages stay on secondary storage.

Working

1. CPU generates an address.
2. OS checks whether the required page is in memory.
3. If present, execution continues.
4. If absent, a **page fault** occurs.
5. OS loads the required page from disk into RAM.

Advantages

- Runs large programs.
- Allows more programs to run simultaneously.
- Improves memory utilization.
- Provides process isolation.

9. Page Fault Handling

Answer

A **page fault** occurs when a required page is not currently present in physical memory.

Steps

1. CPU generates a logical address.
2. Page table is checked.
3. Required page is not found in RAM.
4. Hardware generates a page-fault trap.
5. OS checks whether the reference is valid.
6. OS finds a free frame or selects a page for replacement.
7. Required page is loaded from secondary storage.
8. Page table is updated.
9. Interrupted instruction is restarted.

Thus, the OS brings the required page into memory and continues program execution.

10. Page Replacement and FIFO Algorithm

Answer

When physical memory is full and a new page is required, the OS must remove an existing page. This is called **page replacement**.

Page replacement algorithms include:

- FIFO
- LRU
- Optimal

FIFO – First In First Out

FIFO replaces the page that has been in memory for the **longest time**.

Example

Number of frames = 3

Reference string:

1, 2, 3, 4

Page	Frames	Result
1	1	Page Fault
2	1, 2	Page Fault
3	1, 2, 3	Page Fault
4	4, 2, 3	Page Fault

When page **4** arrives, page **1** is removed because it entered memory first.

Advantages

- Simple to understand.
- Easy to implement.

Disadvantage

FIFO may remove an important page that is still frequently used.

Result: FIFO provides a simple method for managing memory when all frames are occupied.